

Parte Prática – Relatório Técnico

1. Visão Geral da Solução

O projeto consiste na implementação de um MVP (Minimum Viable Product) de uma plataforma distribuída para a Loja Veloz, estruturada com base na arquitetura de microsserviços. A solução foi dividida em quatro serviços principais: API Gateway, Pedidos, Pagamentos e Estoque.

A proposta foi desenvolvida com o objetivo de demonstrar todo o ciclo de entrega de uma aplicação moderna, desde a execução em ambiente local até a implantação automatizada em Kubernetes. Além disso, foram incorporadas práticas relacionadas à padronização do ambiente, segurança básica, estratégias de implantação e observabilidade, garantindo maior confiabilidade e facilidade de manutenção do sistema.

2. Ambiente Local com Docker Compose

Para simplificar a execução da aplicação em ambiente de desenvolvimento, foi utilizado o Docker Compose. Por meio do arquivo `docker-compose.yml`, todos os serviços podem ser iniciados utilizando um único comando (`docker compose up --build`).

A configuração contempla uma rede interna dedicada para comunicação entre os microsserviços, volumes persistentes para armazenamento dos dados do serviço de pedidos e variáveis de ambiente para desacoplar configurações do código-fonte. Essa abordagem favorece a portabilidade e a padronização do ambiente entre diferentes desenvolvedores.

3. Containerização e Versionamento

Cada microsserviço possui seu próprio Dockerfile, implementado utilizando a estratégia de build em duas etapas (multi-stage build), permitindo a redução do tamanho final das imagens e o aumento da segurança.

As imagens utilizam como base o `python:3.12-slim`, instalando apenas as dependências necessárias para execução da aplicação. Além disso, os containers são executados com usuário não privilegiado (non-root) e expõem exclusivamente as portas necessárias para comunicação.

Para o controle de versões, foi adotada a convenção de versionamento semântico, utilizando a tag `1.0.0`. Como evolução futura, essa estratégia poderá ser integrada ao SHA dos commits para proporcionar maior rastreabilidade das versões implantadas.

4. Kubernetes – Produção Mínima

A estrutura de implantação foi organizada na pasta `k8s/base`, contendo os principais manifests necessários para execução da solução em Kubernetes.

Foram implementados recursos como Namespace dedicado, ConfigMap, Secret, Deployments, Services, Horizontal Pod Autoscaler (HPA) e NetworkPolicy. Os Deployments utilizam a estratégia Rolling Update com `maxUnavailable: 0`, reduzindo indisponibilidades durante atualizações.

Além disso, foram configuradas readiness probes e liveness probes para monitoramento da saúde dos containers, bem como requests e limits de CPU e memória para garantir uma utilização adequada dos recursos do cluster.

5. CI/CD

O processo de Integração Contínua e Entrega Contínua (CI/CD) foi implementado utilizando GitHub Actions por meio do workflow localizado em `.github/workflows/ci-cd.yml`.

O pipeline realiza a execução automatizada dos testes em matriz por serviço, a construção das imagens através do Docker Compose, a geração de artefatos contendo os manifests Kubernetes e etapas preparadas para publicação e implantação.

A estrutura foi organizada em fases distintas de teste, build, publicação e deploy, favorecendo a manutenção do fluxo e possibilitando a inclusão de mecanismos de aprovação e controle utilizando o ambiente de produção em futuras evoluções.

6. Observabilidade e Resiliência

A solução foi planejada considerando os três principais pilares da observabilidade: métricas, logs e rastreamento distribuído.

As métricas propostas incluem indicadores como latência, taxa de erro, utilização de CPU, consumo de memória e throughput. Os logs são gerados de forma estruturada em stdout, facilitando a integração com ferramentas de monitoramento.

Para rastreamento distribuído, foi considerada a utilização do OpenTelemetry, permitindo acompanhar o fluxo completo das requisições entre os microsserviços.

Em relação à resiliência, foi adotada a estratégia Rolling Update como abordagem inicial para atualização das aplicações. Como possibilidade de evolução, recomenda-se a adoção de Canary Deploy associado a Service Mesh em serviços críticos.

7. Escalabilidade

Com o objetivo de atender variações de carga e absorver picos de tráfego, foi implementado um Horizontal Pod Autoscaler (HPA) no API Gateway.

A configuração estabelece um mínimo de duas réplicas e um máximo de dez, utilizando o consumo de CPU como métrica inicial de escalonamento. Essa escolha foi realizada considerando que o Gateway representa o principal ponto de entrada das requisições do sistema.

Adicionalmente, foi prevista a possibilidade de adoção futura do Vertical Pod Autoscaler (VPA), permitindo ajustes automáticos mais refinados nos valores de requests e limits dos containers.

8. Infraestrutura como Código com Terraform

A pasta `terraform/` contém a estrutura inicial da abordagem de Infraestrutura como Código (IaC), utilizando o Terraform como ferramenta principal.

Foram definidos componentes básicos, incluindo provider Kubernetes, variáveis, outputs e uma estrutura preparada para futura modularização. O objetivo é permitir que recursos como namespaces, componentes de observabilidade, ingress e mecanismos de segurança sejam provisionados de forma automatizada, padronizada e reproduzível.

Essa abordagem contribui para reduzir erros manuais e aumentar a consistência entre diferentes ambientes.

9. Conclusão

A solução desenvolvida entrega uma base sólida para demonstração acadêmica e validação técnica de um ambiente distribuído baseado em microsserviços.

Embora simplificada para fins educacionais, a implementação contempla os principais requisitos propostos pelo desafio, incluindo padronização do ambiente local, containerização, orquestração com Kubernetes, pipeline CI/CD, estratégias de implantação, escalabilidade automática, configuração desacoplada e fundamentos de observabilidade.

Dessa forma, o projeto demonstra não apenas o funcionamento da aplicação, mas também a aplicação prática de conceitos amplamente utilizados em ambientes corporativos modernos, aproximando o desenvolvimento acadêmico das demandas reais do mercado de tecnologia.